

T1 – Servidor HTTP/1.1 sobre sockets TCP

Livia Noer
Luthero Vargas
Mariana Adam
Nicolas Borges

Angelo Elias Zotto
PUCRS - Escola Politécnica

O trabalho implementa um servidor HTTP/1.1 construído diretamente sobre sockets TCP, utilizando as chamadas `socket`, `bind`, `listen`, `accept`, `recv` e `send`, sem o uso de nenhuma biblioteca HTTP pronta. A implementação foi feita em Python 3, apenas com a biblioteca padrão. O objetivo é observar como o comportamento do TCP, especialmente o custo do 3-way handshake e o RTT da rede, determina o desempenho percebido do HTTP. Para isso, são comparadas conexões novas a cada requisição (cenário C1) com as conexões persistentes do HTTP/1.1 (cenário C2).

As capturas foram feitas com Wireshark entre máquinas distintas da rede local, conectadas via Wi-Fi, e nunca em localhost. A tabela abaixo indica os papéis de cada máquina em cada arquivo de `medicoes/capturas/`. Os papéis se invertem em `teste2clientesWiresharkServer.pcapng`, onde `.221` é o servidor.

Papéis das máquinas em cada captura (endereços 192.168.18.x).

Arquivo	Cliente(s)	Servidor	Usada em
C1.pcapng, C2.pcapng	.221	.220	transação completa; C1 vs C2; overhead; análise do RTT
testes403.pcapng	.221	.220	segurança
t1pacotesespecificos.pcapng	.221 (navegador)	.220	transação completa
teste2clientesWiresharkServer.pcapng	.220 e .219	.221	atendimento simultâneo

A tabela de conformidade foi gerada com o servidor em 127.0.0.1, apenas para registrar o formato exato de cada resposta. A demonstração de segurança vem da captura `testes403.pcapng`, feita entre duas máquinas.

Descrição da arquitetura

Estrutura

O servidor (`src/server.py`) é um único processo Python organizado em três camadas:

- **Aceitação de conexões** (função `serve`): um socket TCP (`AF_INET`, `SOCK_STREAM`) faz `bind` em `0.0.0.0:<porta>`, ou seja, em todas as interfaces, e não apenas em `127.0.0.1`, e entra em um loop de `accept()`. Cada conexão aceita gera uma nova thread (`threading.Thread` com `daemon=True`), que executa a função `handle_connection`, enquanto o loop principal volta imediatamente a chamar `accept()`.
- **Parsing e roteamento por conexão** (`handle_connection`, `read_head`, `parse_request` e `resolve_path`): cada thread atende uma conexão até que ela seja encerrada, seja pelo cliente, pelo header `Connection: close` ou pelo timeout de ociosidade.
- **Montagem da resposta** (`send_response`): monta a linha de status, os headers obrigatórios e o corpo, e envia tudo com `sendall()`.

Parsing da requisição e o fluxo de bytes do TCP

O TCP entrega um fluxo contínuo de bytes, sem respeitar fronteiras de mensagem: uma chamada de `recv()` pode trazer uma requisição incompleta, uma requisição completa ou uma requisição seguida do início da próxima.

A função `read_head` trata isso acumulando os bytes em um buffer até encontrar a sequência `\r\n\r\n`, que marca o fim da seção de cabeçalhos. O que sobra depois desse terminador é preservado e reaproveitado na iteração seguinte do loop de `handle_connection`, correspondendo à próxima requisição na mesma conexão persistente.

A função `skip_body` segue a mesma lógica: usa o valor de `Content-Length` para saber exatamente quantos bytes do corpo devem ser consumidos antes de liberar o restante do buffer para a próxima requisição.

O caminho requisitado é decodificado (*percent-decoding*, função `percent_decode`) antes da verificação de travessia de diretório. Se a decodificação fosse feita depois, uma sequência como `%2e%2e` (a forma codificada de `..`) passaria pela verificação e só se transformaria em `..` depois de já ter sido aprovada.

Estratégia de concorrência: uma thread por conexão

Escolha: uma thread para cada conexão aceita (`threading.Thread`, sem pool de threads).

Justificativa:

O trabalho do servidor em cada conexão é quase inteiramente limitado por I/O: esperar a chegada de bytes via `recv()` e esperar o sistema operacional aceitar os bytes de saída via `send()`. Há pouquíssimo processamento de CPU por requisição (ler um arquivo pequeno do disco e montar uma string de headers). Em cargas desse tipo, o GIL do CPython não é um fator limitante relevante, já que toda chamada bloqueante de socket libera o GIL e permite que outras threads avancem enquanto uma delas está bloqueada.

O volume de conexões simultâneas esperado neste trabalho (testes manuais, poucos clientes e teste de interoperabilidade com outro grupo) é pequeno. A simplicidade de usar uma thread por conexão, sem event loop, callbacks ou uso manual de `select/epoll`, compensa a desvantagem de escalabilidade das threads do sistema operacional, que só se manifesta com milhares de conexões simultâneas. As threads não compartilham estado

mutável (cada uma tem seu próprio socket e buffer; o diretório raiz e as tabelas de status e tipos são somente leitura), então nenhum lock é necessário.

Uma conexão lenta (por exemplo, um cliente que lê a resposta devagar ou que fica ocioso até o timeout) fica isolada em sua própria thread e nunca bloqueia o `accept()` do loop principal nem as demais conexões já aceitas, pois cada thread tem seu próprio socket e seu próprio timeout (`conn.settimeout()`).

Conexões persistentes e timeout de ociosidade (Parte 2)

- **Persistência por padrão:** a função `wants_keep_alive` mantém a conexão aberta em HTTP/1.1, salvo se o header `Connection` contiver `close`. Em HTTP/1.0 a conexão só é mantida se o cliente pedir `keep-alive`.
- **Eco do `Connection: close`:** toda resposta leva `Connection: keep-alive` ou `Connection: close`. No segundo caso, `handle_connection` retorna logo após o envio e o socket é fechado, o que gera o FIN do servidor visto na captura da transação completa.
- **Múltiplas requisições em sequência:** o loop de `handle_connection` atende uma requisição por iteração, sempre reaproveitando os bytes que sobraram no buffer. Assim, até requisições enviadas em um único segmento (*pipelining*) são atendidas em ordem.
- **Timeout de ociosidade:** `conn.settimeout(5)` limita cada `recv()` a 5 segundos. Se nenhum byte chega nesse intervalo, a exceção `socket.timeout` encerra a conexão. O efeito aparece nas capturas: o servidor envia FIN cerca de 5,0 s após a última atividade (ver a captura da transação completa e a evidência de atendimento simultâneo).
- **Requisição malformada:** se o 400 vem da leitura da própria requisição (request line inválida, header sem `:` ou `Content-Length` inválido), a conexão é fechada, já que não é possível saber onde começa a próxima requisição.

Respostas obrigatórias

Toda resposta, incluindo respostas de erro e de HEAD, contém os seguintes headers: `Date`, no formato IMF-fixdate em GMT, gerado manualmente com `time.gmtime()` para não depender do idioma ou locale do sistema; `Server: T1-Lab-Redes/1.0`; `Content-Type`, definido a partir da tabela de extensões `CONTENT_TYPES`, com `application/octet-stream` como valor padrão; e `Content-Length`, com o tamanho exato do corpo. Em requisições HEAD, o `Content-Length` tem o mesmo valor que o GET correspondente teria, mas o corpo não é enviado (parâmetro `head_only` em `send_response`).

Tabela de conformidade

Saída completa em `medicoes/resultados/evidencias/curl_conformidade_e_seguranca.txt`. O checklist automatizado equivalente (`testes/test_server.py`) passou em 8 de 8 casos (`medicoes/resultados/evidencias/test_server_output.txt`).

Código	Comando
200 OK	<code>curl -i http://<ip>:8080/index.html</code>
200 OK (HEAD, mesmos headers, sem corpo)	<code>curl -I http://<ip>:8080/index.html</code>
404 Not Found	<code>curl -i http://<ip>:8080/naoexiste.html</code>
405 Method Not Allowed	<code>curl -i -X DELETE http://<ip>:8080/index.html</code>
400 Bad Request (request line sem versão)	<code>printf 'GET /index.html\r\nHost: <ip>\r\nConnection: close\r\n\r\n' nc <ip> 8080</code>
403 Forbidden	ver a demonstração de segurança, com as três tentativas de travessia

Resposta obtida para cada comando. O corpo de GET /index.html (414 bytes de HTML) foi cortado após a primeira linha:

```
$ curl -i http://<ip>:8080/index.html

HTTP/1.1 200 OK
Date: Sun, 04 Oct 2026 21:33:59 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/html; charset=utf-8
Content-Length: 414
Connection: keep-alive

<!DOCTYPE html>
...

$ curl -I http://<ip>:8080/index.html

HTTP/1.1 200 OK
Date: Sun, 04 Oct 2026 21:33:59 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/html; charset=utf-8
Content-Length: 414
Connection: keep-alive

$ curl -i http://<ip>:8080/naoexiste.html

HTTP/1.1 404 Not Found
Date: Sun, 04 Oct 2026 21:33:59 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 14
Connection: keep-alive

404 Not Found

$ curl -i -X DELETE http://<ip>:8080/index.html

HTTP/1.1 405 Method Not Allowed
```

```
Date: Sun, 04 Oct 2026 21:33:59 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 23
Connection: keep-alive
Allow: GET, HEAD

405 Method Not Allowed

$ printf 'GET /index.html\r\nHost: <ip>\r\nConnection: close\r\n\r\n' | nc <ip>
8080

HTTP/1.1 400 Bad Request
Date: Sun, 04 Oct 2026 21:33:59 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 16
Connection: close

400 Bad Request
```

Demonstração de segurança

O servidor nunca serve um arquivo fora do diretório raiz. A checagem em `resolve_path` rejeita qualquer segmento `..` (depois do percent-decoding) e, como segunda barreira independente, confirma com `os.path.realpath` que o caminho final resolvido (inclusive atravessando symlinks) está dentro de `root`.

As três tentativas abaixo foram feitas com `curl` a partir de 192.168.18.221 contra o servidor em 192.168.18.220:8080 e estão em `medicoes/capturas/testes403.pcapng`. Todas tentam ler o próprio código do servidor (`src/server.py`), que fica fora de `www/`, em um diretório irmão. As requisições e respostas foram copiadas dos pacotes da captura. A opção `--path-as-is` impede que o `curl` normalize o `..` antes de enviar: o servidor precisa rejeitar o caminho que efetivamente chega pela rede.

1. Travessia direta (frames 4 e 6):

```
GET ../../src/server.py HTTP/1.1
Host: 192.168.18.220:8080
User-Agent: curl/8.21.0
Accept: */*

HTTP/1.1 403 Forbidden
Date: Sun, 04 Oct 2026 05:53:23 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 14
Connection: keep-alive

403 Forbidden
```

2. Percent-encoding em .. (%2E%2E; frames 13 e 15):

```
GET /%2E%2E/src/server.py HTTP/1.1
Host: 192.168.18.220:8080
User-Agent: curl/8.21.0
Accept: */*

HTTP/1.1 403 Forbidden
Date: Sun, 04 Oct 2026 05:53:23 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 14
Connection: keep-alive

403 Forbidden
```

3. Percent-encoding em .. e nas barras (%2E%2E%2F; frames 22 e 24):

```
GET /%2E%2E%2Fsrc%2Fserver.py HTTP/1.1
Host: 192.168.18.220:8080
User-Agent: curl/8.21.0
Accept: */*

HTTP/1.1 403 Forbidden
Date: Sun, 04 Oct 2026 05:53:23 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 14
Connection: keep-alive

403 Forbidden
```

As três tentativas resultam em 403 Forbidden, sem expor nenhum conteúdo fora de `www/`. Nos casos 2 e 3, o `..` só aparece depois da decodificação, o que confirma que a checagem é feita sobre o caminho já decodificado.

Captura de uma transação completa

A figura abaixo mostra um `GET /index.html` completo feito de outra máquina: a primeira conexão de `medicoes/capturas/C1.pcapng` (cliente 192.168.18.221, servidor 192.168.18.220:8080), isolada com o filtro `tcp.stream eq 0`. A tabela em seguida identifica cada fase.

tcp.stream eq 0						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.18.221	192.168.18.220	TCP	66	63498 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.210523800	192.168.18.220	192.168.18.221	TCP	66	8080 → 63498 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM WS=512
3	0.210667700	192.168.18.221	192.168.18.220	TCP	54	63498 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4	0.210930400	192.168.18.221	192.168.18.220	HTTP	123	GET /index.html HTTP/1.1
5	0.215557600	192.168.18.220	192.168.18.221	TCP	60	8080 → 63498 [ACK] Seq=1 Ack=70 Win=65536 Len=0
6	0.219823600	192.168.18.220	192.168.18.221	HTTP	630	HTTP/1.1 200 OK (text/html)
7	0.219824600	192.168.18.220	192.168.18.221	TCP	60	8080 → 63498 [FIN, ACK] Seq=577 Ack=70 Win=65536 Len=0
8	0.219918700	192.168.18.221	192.168.18.220	TCP	54	63498 → 8080 [ACK] Seq=70 Ack=578 Win=64768 Len=0
9	0.220123500	192.168.18.221	192.168.18.220	TCP	54	63498 → 8080 [FIN, ACK] Seq=70 Ack=578 Win=64768 Len=0
11	0.223061900	192.168.18.220	192.168.18.221	TCP	60	8080 → 63498 [ACK] Seq=578 Ack=71 Win=65536 Len=0

Wireshark: `GET /index.html` completo entre duas máquinas (`C1.pcapng`, filtro `tcp.stream eq 0`).

Fases da transação (tempos relativos ao primeiro pacote).

Frame	Tempo (s)	Pacote	Fase
1	0,000000	SYN (cliente → servidor)	Handshake TCP
2	0,210524	SYN, ACK	Handshake TCP
3	0,210668	ACK	Handshake TCP
4	0,210930	PSH, ACK, 69 B de payload: GET /index.html HTTP/1.1	Requisição
5	0,215558	ACK do servidor	Confirmação da requisição
6	0,219824	PSH, ACK, 576 B de payload: HTTP/1.1 200 OK (162 B de headers + 414 B de corpo)	Resposta
7	0,219825	FIN, ACK do servidor	Encerramento
8	0,219919	ACK do cliente	Encerramento
9	0,220124	FIN, ACK do cliente	Encerramento
11	0,223062	ACK do servidor	Encerramento

Como a requisição do cenário C1 leva `Connection: close`, o servidor responde com o mesmo header e fecha a conexão logo após a resposta (frame 7, 1 μ s depois do frame 6). O encerramento tem as 4 etapas completas: FIN do servidor, ACK do cliente, FIN do cliente e ACK do servidor. O frame 10 não aparece porque pertence à conexão seguinte (é o SYN da conexão 2). A demora de 210 ms entre os frames 1 e 2 é discutida no item RTT medido.

Complemento: navegador e conexão persistente. A captura `medicoes/capturas/t1pacotesespecificos.pcapng` mostra o navegador do cliente .221 carregando a página de teste, que referencia `style.css`, `rede.png`, `app.js` e `dados.json`:

- GET / (frame 4) e GET /style.css (frame 8) reutilizam a mesma conexão (stream 0), sem novo handshake.
- O navegador abre mais duas conexões em paralelo (handshakes nos frames 9–14) para buscar `rede.png` e `app.js` ao mesmo tempo, e reutiliza uma delas para GET /dados.json (frame 23).
- A resposta de `rede.png` (20 998 B de corpo) aparece em três registros (frames 21, 24 e 25, com 14 600, 5 840 e 712 B de payload). Como 14 600 B excede o MSS de 1 460 B, o driver de rede do cliente (Windows) agregou vários segmentos recebidos antes de entregá-los à captura (*Receive Segment Coalescing*); no fio foram cerca de 15 segmentos.
- O navegador não envia `Connection: close`. As três conexões ficam ociosas após $t \approx 0,56$ s e o servidor as fecha pelo timeout: os FIN, ACK dos frames 30, 32 e 34 saem em $t \approx 5,51$ s, cerca de 5,0 s após a última atividade de cada conexão, e o cliente confirma com ACK (frames 31, 33 e 35). O navegador não envia seu próprio FIN antes do fim da captura, então ali aparece apenas a metade do encerramento iniciada pelo servidor.

Evidência de atendimento simultâneo

Em `medicoes/capturas/teste2clientesWiresharkServer.pcapng`, o servidor `192.168.18.221:8080` atende dois clientes distintos, `192.168.18.220` (cliente A) e `192.168.18.219` (cliente B), que carregam a mesma página de teste ao mesmo tempo. Requisições e respostas HTTP em ordem cronológica (tempo relativo em segundos; extraído de `medicoes/resultados/evidencias/teste2clientes_http.txt`):

0,0057	cliente A -> servidor	GET /
0,0073	servidor -> cliente A	200 /
0,0279	cliente B -> servidor	GET /
0,0289	servidor -> cliente B	200 /
0,0365	cliente A -> servidor	GET /style.css
0,0365	cliente B -> servidor	GET /style.css
0,0379	servidor -> cliente A	200 /style.css
0,0380	servidor -> cliente B	200 /style.css
0,0499	cliente B -> servidor	GET /rede.png
0,0499	cliente B -> servidor	GET /app.js
0,0514	servidor -> cliente B	200 /app.js
0,0559	cliente A -> servidor	GET /rede.png
0,0571	servidor -> cliente B	200 /rede.png
0,0583	cliente B -> servidor	GET /dados.json
0,0589	cliente A -> servidor	GET /app.js
0,0596	servidor -> cliente B	200 /dados.json
0,0597	servidor -> cliente A	200 /app.js
0,0735	servidor -> cliente A	200 /rede.png
0,1115	cliente A -> servidor	GET /dados.json
0,1122	servidor -> cliente A	200 /dados.json

As requisições dos dois clientes chegam intercaladas: os GET `/style.css` de A e B (frames 13 e 14) chegam com apenas 3 μ s de diferença, e os dois recebem 200 OK em seguida. As seis conexões TCP abertas pelos dois navegadores ficam ativas ao mesmo tempo (tabela abaixo). Nenhum cliente esperou o outro terminar. As seis conexões terminam por volta de 5 s, quando o servidor as fecha pelo timeout de ociosidade.

Conexões TCP em `teste2clientesWiresharkServer.pcapng`.

Cliente (IP:porta)	Início (s)	Duração (s)
A – 192.168.18.220:50676	0,0000	5,14
B – 192.168.18.219:50648	0,0119	5,07
B – 192.168.18.219:50649	0,0377	5,04
B – 192.168.18.219:50650	0,0377	5,04
A – 192.168.18.220:50680	0,0473	5,12
A – 192.168.18.220:50684	0,0473	5,10

RTT medido

O RTT foi medido com ping a partir do cliente (`192.168.18.221`), com saída do Windows reproduzida sem cortes:

```
> ping -n 10 192.168.18.220
```

Disparando 192.168.18.220 com 32 bytes de dados:

```
Resposta de 192.168.18.220: bytes=32 tempo=78ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=88ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=105ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=103ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=110ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=115ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=32ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=39ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=42ms TTL=64
Resposta de 192.168.18.220: bytes=32 tempo=4ms TTL=64
```

Estatísticas do Ping para 192.168.18.220:

Pacotes: Enviados = 10, Recebidos = 10, Perdidos = 0 (0% de perda),

Aproximar um número redondo de vezes em milissegundos:

Mínimo = 4ms, Máximo = 115ms, Média = 71ms

O RTT também pode ser medido na própria captura, pelo intervalo entre o SYN do cliente e o SYN, ACK do servidor (tabela abaixo).

RTT por fonte.

Fonte	Valor	Observação
ping (10 amostras)	mín. 4 ms, máx. 115 ms, média 71 ms	alta variação entre amostras
SYN → SYN-ACK, conexões 2–10 do C1	3,0–11,2 ms; média 5,1 ms , mediana 4,1 ms	RTT de regime durante a medição
SYN → SYN-ACK, 1ª conexão do C1	210,5 ms	primeira conexão da sessão
SYN → SYN-ACK, conexão única do C2	101,1 ms	primeira conexão da sessão
SYN → SYN-ACK, 1ª conexão de t1pacotesespecificos	81,4 ms	primeira conexão da sessão

A rede é Wi-Fi, e o ping mostra a variação típica desse meio: amostras de 4 a 115 ms na mesma série, com valores altos espalhados e não só na primeira. Um efeito diferente aparece nas capturas: a primeira conexão de cada sessão leva de 81 a 210 ms para receber o SYN, ACK, enquanto as seguintes levam de 3 a 11 ms. Hipótese: o adaptador Wi-Fi sai do modo de economia de energia ou a resolução ARP ainda não está em cache (o filtro tcp.port não captura ARP, então não foi possível confirmar).

Na análise em função do RTT é usado o **RTT de regime de 5,1 ms**, medido durante a própria sessão C1/C2. A média do ping (71 ms) não representa a rede no momento da medição: com ela, os 9 handshakes extras do C1 deveriam custar $9 \times 71 = 639$ ms, mas toda a diferença medida entre C1 e C2 foi de 177 ms.

Tabela comparativa C1 vs C2

Métricas extraídas com tshark/capinfos de medicoes/capturas/C1.pcapng e C2.pcapng (pacote a pacote em medicoes/resultados/evidencias/c1_pacotes.txt e c2_pacotes.txt; totais em c1_capinfos.txt e c2_capinfos.txt). Servidor 192.168.18.220:8080, cliente 192.168.18.221, 10 requisições sequenciais a /index.html em cada cenário. A captura foi feita no cliente. Os bytes são o tamanho dos frames Ethernet registrados pelo Wireshark: os frames do servidor aparecem com 60 B (mínimo Ethernet, com preenchimento) e os do cliente com 54 B, porque são capturados antes do preenchimento.

Métrica	C1 — conexão nova por requisição	C2 — conexão persistente
Handshakes TCP completos	10	1
Pacotes totais	100	28
Bytes totais (frames Ethernet)	12 270	8 094
Tempo total	376,99 ms	200,20 ms

Economia da conexão persistente:

Métrica	Economia (C2 vs C1)
Pacotes	72,0 % menos pacotes
Bytes	34,0 % menos bytes
Tempo total	46,9 % mais rápido

Análise do overhead de conexão

Cada uma das 10 conexões do C1 segue o mesmo padrão, verificado pacote a pacote: **handshake** (SYN 66 B + SYN-ACK 66 B + ACK 54 B = 186 B, 3 pacotes) e **encerramento** (FIN-ACK 60 B + ACK 54 B + FIN-ACK 54 B + ACK 60 B = 228 B, 4 pacotes), ou seja, **7 pacotes e 414 bytes por conexão** sem nenhum byte de dado de aplicação.

	Overhead (abrir+fechar TCP)	Total do cenário C1	% do total
Pacotes	70	100	70,0 %
Bytes	4 140	12 270	33,7 %

No C1, **70 % dos pacotes não carregam nenhum dado de aplicação**: servem só para abrir e fechar conexões TCP. Apenas 52,6 % dos bytes do C1 são HTTP (6 450 de 12 270 B); no C2, essa fração sobe para 80,8 % (6 540 de 8 094 B). No C2, o custo de abrir e fechar acontece uma única vez para as 10 requisições (7 pacotes, 414 bytes).

A diferença entre os cenários fecha exatamente com esses números:

- **Pacotes:** $100 - 28 = 72 = 9 \times 7 + 9$. São 63 pacotes de abertura e fechamento das 9 conexões extras, mais 9 ACKs avulsos: no C1 o servidor confirma cada GET com um ACK separado, enquanto no C2 essa confirmação vai junto com a resposta.

- **Bytes:** $12\,270 - 8\,094 = 4\,176 = 9 \times 414 + 9 \times 60 - 90$. O desconto de 90 B vem do próprio C2: o header Connection: keep-alive tem 5 caracteres a mais que Connection: close e aparece em 9 requisições e 9 respostas.

Análise em função do RTT

Modelo. Toda conexão nova paga 1 RTT de handshake antes de poder enviar a requisição: o SYN parte e o SYN-ACK só chega 1 RTT depois (o ACK final não espera resposta, então não soma RTT). Depois disso, cada requisição sequencial custa mais 1 RTT (requisição e resposta). Assim:

- C1: 10 conexões $\times$ (1 RTT de handshake + 1 RTT de requisição) = **20 RTT**;
- C2: 1 RTT de handshake + 10 RTT de requisições = **11 RTT**;
- o C1 gasta **9 RTTs a mais**, um para cada handshake das 9 conexões extras.

O encerramento não soma RTT ao tempo do cliente: ele envia o SYN da conexão seguinte logo depois do próprio FIN (0,8 ms entre os frames 9 e 10 do C1), sem esperar o fim do fechamento. O custo do encerramento aparece em pacotes (4 por conexão), não em tempo.

Medição. A diferença total entre os cenários é $376,99 - 200,20 = 176,79$ ms. Medindo cada parcela na captura (tabela abaixo):

Decomposição da diferença de tempo entre C1 e C2.

Parcela	ms	Origem
9 handshakes extras (SYN $\rightarrow$ SYN-ACK das conexões 2–10 do C1)	46,10	custo estrutural de não usar conexão persistente
Primeiro handshake mais lento no C1 (210,52 contra 101,06 ms)	109,46	efeito de primeira conexão (ver RTT medido), não causado pela persistência
Respostas mais lentas no C1 (soma de GET $\rightarrow$ resposta: 90,78 contra 72,07 ms)	18,71	provavelmente por cada requisição chegar em uma conexão recém-aberta
Intervalo entre o ACK do handshake e o envio do GET (11,58 contra 9,13 ms)	2,45	lado do cliente
Outros	0,07	
Total	176,79	

Quantos RTTs a mais. Os 9 handshakes extras somam 46,10 ms. Cada handshake custa, por definição, 1 RTT (SYN $\rightarrow$ SYN-ACK), então o C1 gasta **9 RTTs a mais, que vêm inteiramente do 3-way handshake de cada conexão nova**, como o modelo prevê. Em média, cada um desses RTTs durou $46,10/9 = 5,1$ ms, que é o RTT de regime medido no item RTT medido. O encerramento das conexões não acrescenta tempo: em todas as conexões do C1, o SYN seguinte sai menos de 1 ms após o FIN do cliente.

A maior parcela medida (109,46 ms) não é um custo da conexão não persistente: é a diferença entre as primeiras conexões das duas sessões, ambas lentas por efeito do Wi-Fi. Descontando a espera do primeiro SYN-ACK de cada cenário, os tempos ficam em

166,46 ms (C1) e 99,14 ms (C2), uma razão de 1,68 entre os cenários, próxima da razão $20/11 \approx 1,8$ prevista pelo modelo.

Conclusão

Para N requisições sequenciais ao mesmo servidor, a conexão persistente do HTTP/1.1 elimina $N - 1$ handshakes TCP, cada um custando 1 RTT, além de 7 pacotes e 414 bytes de abertura e fechamento por conexão. Nas medições:

- 72 % menos pacotes e 34 % menos bytes no C2; no C1, 70 % dos pacotes não carregam dado de aplicação;
- 9 RTTs a mais no C1 (46 ms com RTT de regime de 5,1 ms), todos vindos do handshake das conexões novas.

O ganho em tempo é aproximadamente (número de conexões evitadas) $\times$ RTT, então ele seria **maior em uma rede com RTT maior**. Os mesmos 9 handshakes custaram 46 ms nesta rede local; com RTT de 100 ms (típico de um servidor distante na internet), custariam cerca de 0,9 s, enquanto o C2 continuaria pagando um único handshake. O ganho também cresce com o número de objetos por página (mais conexões evitadas) e com arquivos maiores, pois cada conexão nova recomeça o slow start do TCP. Se a conexão usasse TLS (HTTPS), cada conexão nova pagaria ainda 1 ou 2 RTTs a mais de negociação.

A economia percentual de **bytes**, por outro lado, depende do tamanho das respostas: os 414 B de controle por conexão representam 34 % aqui porque `index.html` tem só 414 B de corpo, mas seriam cerca de 2 % para um arquivo de 20 KB.